# Algoritmos e Programação: Orientação a Objetos

## Introdução à Programação em C++

**Profa. Andriele Busatto do Carmo**

acarmo@unisinos.br

UNISINOS

# Linguagem C++

- Histórico
  - Desenvolvida por Bjarne Stroustrup.
  - Primeiro release de 1985, como uma extensão da linguagem C.
  - Projetada para programação de sistemas, software embarcado com recursos limitados e grandes sistemas.
    - Com foco em desempenho, eficiência e flexibilidade de uso.
  - Padronizada pela ISO (*International Organization for Standardization*)
    - Última versão do padrão: 2020 – C++20.

# Linguagem C++

- Principais características
  - Orientada a objetos
  - Linguagem compilada
  - Portável
  - Recursos de mais “baixo nível”
  - Geração de código eficiente
  - Foco em desempenho
  - Possibilidade de uso para propósitos variados
    - Desenvolvimento de software básico (Sistemas Operacionais, compiladores, etc).

# Compilador GCC (GNU *Compiler Collection*)

- Compilador que utilizaremos para os códigos escritos na linguagem C++.

- Comando para compilação via terminal:
  - g++ -o arquivo_executavel arquivo_codigo_fonte.cpp

# Estrutura básica de um programa em C++

```
// Estrutura de um programa em C++


ARQUIVOS DE CABEÇALHO

•  headers

•  namespaces


DEFINIÇÃO DE CONSTANTES, TIPOS
    ESPECÍFICOS, ETC (define)


DEFINIÇÃO DE VARIÁVEIS GLOBAIS


DEFINIÇÃO DE FUNÇÕES (incluindo a
    main)
```

```
// Exemplo de um programa em C++


#include <iostream>


#using namespace std;


#define VALUE 5


int main() {

    int a;              //declara a variável a

    a = 4 + VALUE;      //soma 4 com 5 (VALUE)


    return 0;

}
```

# C++ namespaces

- Namespaces definem escopo:
  - Espaço em que são definidos ou declarados identificadores (variáveis, métodos, classes).

- Com namespaces é possível definir o espaço ou contexto no qual identificadores são definidos.

- Vantagem: evitar colisão de identificadores.

# C++ namespaces

- Namespace: **std**
  - Identificadores da biblioteca padrão do C++.

- Três formas de uso:
  - Operador: **::** (2x dois pontos)
  - Declarador: **using**
  - Diretiva: **using**

# C++ namespaces

- Uso com o operador **::**

```
#include <iostream>

int main() {
    std::string nome;
    std::cout << "Informe o seu nome: ";
    std::cin >> nome;
    std::cout << "Olá " << nome << "!" << std::endl;
    std::cout << "Seja bem-vindo(a)!";

  return 0;
}
```

# C++ namespaces

- Uso com declarador:

```
#include <iostream>

// declaração para cout, endl e string
using std::cout;
using std::endl;
using std::string;

int main() {
    string nome;
    cout << "Informe o seu nome: ";
    std::cin >> nome;
    cout << "Olá " << nome << "!" << endl;
    cout << "Seja bem-vindo(a)!";

    return 0;
}
```

# C++ namespaces

- Uso com diretiva:

```
#include <iostream>

// usando diretiva
using namespace std;

int main() {
    string nome;
    cout << "Informe o seu nome: ";
    cin >> nome;
    cout << "Olá " << nome << "!" << endl;
    cout << "Seja bem-vindo(a)!";

    return 0;
}
```

# Entrada/Saída com *Streams*

# Saída com *Streams*

- Objeto "***cout***"
  - Representa o *stream* de saída no C++.
    - *Streams* representam fluxo de entrada e saída de dados. Neste caso, um fluxo (sequência) de dados a serem impressos na tela.
  - Para "imprimir" na tela, faz-se uso do *operador de inserção* representado por "<<". Este operador "insere" dados no *stream*.

# Saída com *Streams*

- Operador de inserção: **<<**
  - Operador sobrecarregado que executa a saída com *streams* em C++ (imprime na tela).
  - O objeto ***cout*** é usado em conjunto com ele para a impressão de dados.

# Saída com *Streams*

- Exemplo de uso:

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World!\n";

    // impressão de uma linha com múltiplos comandos
    cout << "Teste com ";
    cout << "dois couts\n";

    // manipulador endl → gera um caractere nova linha
    // Além disso, descarrega o buffer de saída
    cout << "Escrevendo uma linha..." << endl;

    cout << "Mais uma vez...\n";

    return 0;
}
```

# Saída com *Streams*

- O operador de inserção **<<** sempre retorna uma referência ao objeto **cout**. Desta forma, é possível encadear vários **cout**, o que perrmite combinar constantes e variáveis.

- Exemplo:

```
int x = 15;
int y = 20;
// imprime uma string e o resultado da soma entre as variáveis x e y
cout << "Saída encadeadas: soma de dois números " << x + y << endl;
```

# Entrada com *Streams*

- Objeto "***cin***"
  - Representa o *stream* de entrada no C++.
  - Realiza a leitura de uma sequência de dados, sem espaços e sem tabulações, vindas do teclado.
  - Para ler esses dados (coletá-los do *stream*), faz-se o uso do *operador de extração* representado por ">>". Este operador "extrai" dados do *stream*.

# Entrada com *Streams*

- Operador de extração: **>>**
  - Operador sobrecarregado que executa a entrada com *streams* em C++.
  - O objeto ***cin*** é usado em conjunto com ele para leitura dos dados.
    - Variáveis podem ser usadas para o armazenamento das informações.

# Entrada com *Streams*

- Exemplo de uso:

```
#include <iostream>
using namespace std;
int main() {
    int valor1;
    int valor2;

    cout << "Leitura do primeiro valor: ";
    cin >> valor1;
    cout << endl;
    cout << "Leitura do segundo valor: ";
    cin >> valor2;
    cout << endl;

    return 0;
}
```

# Entrada com *Streams*

- Da mesma forma que o operador de inserção, o operador de extração **>>** retorna uma referência ao objeto **cin**, o que permite encadear vários **cin**.

- Exemplo:

```
float x;
float y;

// entrada de um float, pressiona <ENTER>, entrada de outro float, pressiona <ENTER>
cin >> x >> y;
```

# Entrada/Saída seguras quanto ao tipo

- Dados inesperados podem ser processados pelos operadores **<<** e **>>**. Por isso, indicadores de erros podem ser ativados para indicar se houve sucesso ou falha em uma operação de entrada/saída.

# Manipuladores de *Streams*

- Utilizados para formatação. Alguns recursos em destaque:

  - **Base do *stream* de inteiros**:

```
#include <iomanip>
#include <iostream>
using namespace std;
//...
cin >> value;
cout << hex << value << endl          // apresenta value em hexadecimal
     << oct << value << endl          // apresenta value em octadecimal
     << setbase(10) << value << endl; // apresenta value na base 10
//...
```

# Manipuladores de *Streams*

- Recursos de formatação em destaque:
  - **Precisão em ponto flutuante**:

```
double PI = 3.14159265;


cout.precision(5); // define a precisão
cout << setiosflags(ios::fixed) << PI << endl;// imprime PI com 5 casas decimais
OU
// define e imprime PI com 5 casas decimais
cout << setiosflags(ios::fixed) << setprecision(5) << PI;
```

# Manipuladores de *Streams*

- Recursos de formatação em destaque:
  - **Largura de campo**:
    - Para definir a largura da impressão de uma variável usa-se o **setw(n)**, onde n é número de caracteres desejado para impressão desta variável. A função só afeta a próxima variável do **cout**.
    - Exemplo:

```
int caixas = 45;

cout << "Número de caixas: " << setw(10) << caixas << endl;
```

# Manipuladores de *Streams*

- Recursos de formatação em destaque:
  - **Alinhamento de impressão**:
    - É possível alinhar os dados impressos tanto à esquerda quanto à direita, com manipuladores **left** e **right**.
    - Exemplo:

```
cout  << right <<  10;
cout  << left <<  10;
```

# Entrada de *Strings*

- Em algumas ocasiões pode ser necessária a coleta de dados que contenham *strings* com tabulação, espaços em branco e/ou novas linhas.
  - Frases são exemplos deste tipo de situação. No entanto, o operador de extração **>>** ignora tais caracteres.

- O C++ oferece então a função **getline**. Esta função remove o delimitador do *stream* (ou seja, lê o caractere e o descarta) e armazena o mesmo em uma variável definida pelo usuário.

# Entrada de *Strings*

- Exemplo de uso da função **getline**:

```
#include <iostream>
using namespace std;
int main() {
    const SIZE = 100;
    char armazena[SIZE];

    cout << "Digite uma frase com espaços: " << endl;
    cin.getline(arrmazena, SIZE);
    cout << "A frase digitada é: \n" << armazena << endl;

    return 0;
}
```